



7. Programació Orientada a Objectes

RA2, RA4, RA7

MP0485 - Programació

Curs 2025-2026



1. Programació Orientada a Objectes

La **programació orientada a objectes** és un paradigma (model) que canvia la manera d'estructurar el codi, fent servir **objectes** per representar les dades.

Les dades i operacions s'organitzen en estructures reutilitzables anomenades **classes**.

➔ Les **classes** permeten crear **objectes** concrets (instàncies).

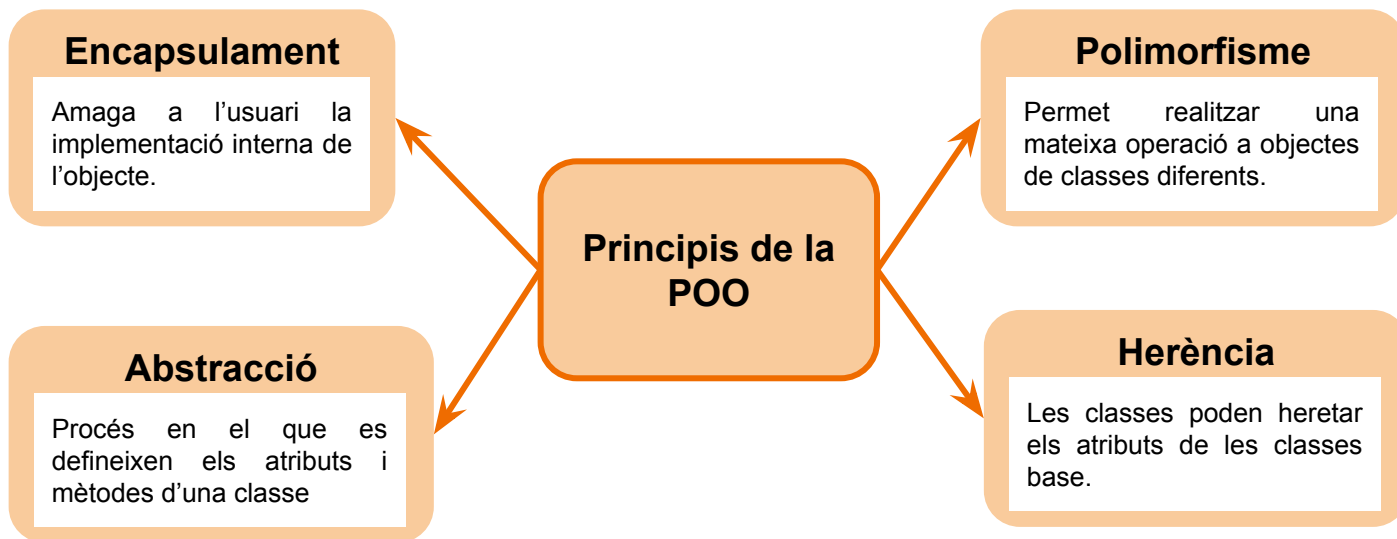
Els **llenguatges de programació** poden ser orientats a objecte (POO) o no. Hi ha diversos paradigmes i llenguatges que n'admeten més d'un.

➤
➤
Llenguatges de POO: Java, C++, Python, PHP...

Llenguatges de PP: C, Pascal, Fortran, COBOL...



2. Els 4 principis de la POO





3. Classes i objectes

Una **classe** guarda una abstracció del que representa un objecte. És com una *plantilla* que **defineix com seran els objectes** d'un tipus concret.

Amb una classe podem instanciar tants objectes com vulguem.

Què caracteritza un cotxe? (Procés d'abstracció)





3. Classes i objectes

Una **classe** guarda una abstracció del que representa un objecte. És com una *plantilla* que **defineix com seran els objectes** d'un tipus concret.

Amb una classe podem instanciar tants objectes com vulguem.

Què caracteritza un cotxe? (Procés d'abstracció)

- marca
- model
- color
- nombre de portes
- nombre de places
- descapotable
- ...





4. Atributs i mètodes

Les **classes** consten dels **següents elements**:

1. **Atributs**: són les característiques de l'objecte.
2. **Mètodes constructors**: permeten crear una instància (objecte) de la classe, amb uns valors concrets.
3. **Mètodes getters i setters**: permeten obtenir o canviar el valor d'un atribut.
4. **Altres mètodes**: podem crear mètodes específics per a una classe.



5. Objectes

Com hem dit abans, **un objecte és una instància d'una classe.**

Un cotxe es caracteritza per...

- marca
- model
- color
- nombre de portes
- nombre de places
- descapotable
- ...



Com és aquest cotxe en concret?



5. Objectes

Com hem dit abans, **un objecte és una instància d'una classe.**

Un cotxe es caracteritza per...

- marca
- model
- color
- nombre de portes
- nombre de places
- descapotable
- ...



**Com és aquest
cotxe en concret?**

gris
5 portes
5 places
no descapotable



6. Estructura d'una classe

`class Cotxe:` → **Nom de la classe**

Constructor

```
def __init__(self, color, num_portes, num_places, descapotable, propietari):  
    self.color = color  
    self.num_portes = num_portes  
    self.num_places = num_places  
    self.descapotable = descapotable  
    self.__propietari = propietari
```

→ **Atributs**

→ **Constructor**

La variable "**self**" fa referència a la instància del propi objecte (la variable que el guarda).

Getter per obtenir el propietari

```
def get_propietari(self):  
    return self.__propietari
```

→ **Mètode getter**

Getter per obtenir si és descapotable

```
def descapotable(self):  
    return self.descapotable
```

→ **Mètode getter d'un boolean**

Setter per establir el propietari

```
def set_propietari(self, propietari):  
    self.__propietari = propietari
```

→ **Mètode setter**

Setter per establir si és descapotable

```
def descapotable(self, descapotable):  
    self.descapotable = descapotable
```

→ **Mètode setter d'un boolean**

Altres getters i setters...



7. Atributs públics i privats

Atributs públics:

- Es poden accedir i modificar directament.

Exemple:

```
cotxe1.num_portes=5  
print(cotxe1.num_portes)
```

Els **atributs** poden ser públics o privats. Els atributs públics es poden accedir i modificar directament des de l'exterior de la classe o objecte, mentre que els atributs privats requereixen mètodes específics per a l'accés i la modificació

Atributs privats:

- El nom ha de començar amb doble guió baix (__nom).
- Només es poden accedir i modificar amb els getters i setters.

Exemple:

```
cotxe1.set_color("Vermell")  
print(f"Nou color del cotxe: {cotxe1.get_color()}")
```

Inicialment, prenen els valors passats al constructor.

Atributs protegits:

- El nom ha de començar amb un guió baix (_nom).
- Les classes derivades de la classe hi poden accedir directament, però la resta ho han de fer des del getter i setter.



7. Atributs públics i privats

Exemple atribut públic

```
# Crear un objecte de la classe Cotxe
cotxe1 = Cotxe(color="Blau", num_portes=4, num_places=5,
descapotable=False, propietari="Joan")

# Accés i modificació de l'atribut num_portes
print(f"Numero de portes del cotxe: {cotxe1.num_portes}")
cotxe1.num_portes=5
print(f"Numero de portes del cotxe: {cotxe1.num_portes}")
cotxe1.set_num_portes(6)
print(f"Numero de portes del cotxe: {cotxe1.num_portes}")
```

```
Numero de portes del cotxe: 4
Numero de portes del cotxe: 5
Numero de portes del cotxe: 6
```

Exemple atribut privat

```
# Crear un objecte de la classe Cotxe
cotxe1 = Cotxe(color="Blau", num_portes=4, num_places=5,
descapotable=False, propietari="Joan")

# Accés i modificació de l'atribut color
print(f"Propietari del cotxe: {cotxe1.get_propietari()}")
cotxe1.__propietari="Pere"
print(f"Propietari del cotxe: {cotxe1.get_propietari()}")
cotxe1.set_propietari("Pere")
print(f"Nou propietari del cotxe: {cotxe1.get_propietari()}")
```

```
Propietari del cotxe: Joan
Propietari del cotxe: Joan
Nou propietari del cotxe: Pere
```



7. Exemple: objecte cotxe de l'Àlex

```
def demanar_dades():
```

```
    propietari = input("Com et dius? ")
```

```
    color = input("De quin color és el cotxe? ")
```

```
    numportes = int(input("Quantes portes té? "))
```

```
    numplaces = int(input("Quantes places té? "))
```

```
    d = input("És descapotable? (S/N) ")
```

```
    descapotable = True if d.upper() == "S" else False
```

1. Demanem els
valors per teclat

```
    cotxe = Cotxe(color, numportes, numplaces, descapotable, propietari)
```

```
    return cotxe
```

2. Creem el cotxe amb els valors

```
# Codi principal
```

```
if __name__ == "__main__":
```

```
    cotxe1 = demanar_dades()
```

```
    print(cotxe1)
```

3. Assignem a cotxe1 el cotxe creat

Hem fet el següent codi per
guardar un cotxe.

**Ara com puc imprimir les
dades?**



7. Exemple: objecte cotxe de l'Àlex

```
print(cotxe1)
```

```
<__main__.Cotxe object at 0x000002BA0415ACD0>
```

Hem fet el següent codi per
guardar un cotxe.

**Ara com puc imprimir les
dades?**



8. Mètode `__str__()`

```
def __str__(self):  
    return f'''Cotxe:  
    Color: {self.color}  
    Portes: {self.num_portes}  
    Places: {self.num_places}  
    Descapotable: {'Sí' if self.descapotable else 'No'}  
    Propietari: {self.__propietari}'''  
  
print(cotxe1)
```

Cotxe:

Color: Blau
Portes: 4
Places: 5
Descapotable: No
Propietari: Àlex

El mètode `__str__` és un mètode propi de qualsevol objecte i que es pot **sobreescriure** en una classe per tal d'imprimir l'objecte tal i com nosaltres volguem.

Permet que quan fem **print(objecte)** ens imprimeixi les dades.



9. Mètodes propis

També podem definir **mètodes propis** per a una classe.

Ampliarem la classe cotxe per tal de poder guardar el consum per cada 100 quilòmetres d'un cotxe, i així poder calcular la despesa en €.

```
class Cotxe:
    (...)
    def calcular_despesa(self, preu_combustible):
        return self.consum*preu_combustible

print(f"El teu cotxe gasta {cotxe1.calcular_despesa(2)}€ per cada 100km")
```

El teu cotxe gasta 40€ per cada 100km



10. Gestió de més d'un objecte

Ara volem ampliar el programa per tal de **gestionar diversos cotxes**. Ho podem fer **creant una llista** com fem habitualment, només que aquesta vegada guardarà **objectes de tipus Cotxe**.

```
# Codi principal
if __name__ == "__main__":
    cotxes = [] 1. Creem una llista buida.
    continuar = input("Vols afegir un nou cotxe? S/N").upper()
    while (continuar=="S"):
        nou_cotxe = demanar_dades() 2. Creem un nou cotxe amb el mètode creat abans.
        cotxes.append(nou_cotxe) 3. Afegim el nou cotxe a la llista per guardar-lo.
        print(f"S'ha afegit un nou cotxe (nom del propietari: {nou_cotxe.get_propietari()}).")
        continuar = input("Vols afegir un nou cotxe? S/N").upper()
```

```
S'ha afegit un nou cotxe (nom del propietari: Pepa).
S'ha afegit un nou cotxe (nom del propietari: Joan).
S'ha afegit un nou cotxe (nom del propietari: Clara).
```

En sortir de l'*if* el cotxe s'esborrarà de la variable "cotxe", però no de la llista. En tornar-lo a executar, per tant, podrem crear un altre cotxe i guardar-lo a la llista.



10. Gestió de més d'un objecte

```
for cotxe in cotxes:  
    print(cotxe)
```

```
Cotxe:  
    Color: Rosa  
    Portes: 2  
    Places: 2  
    Descapotable: Sí  
    Propietari: Pepa  
  
Cotxe:  
    Color: Negre  
    Portes: 4  
    Places: 5  
    Descapotable: No  
    Propietari: Joan  
  
Cotxe:  
    Color: Blanc  
    Portes: 5  
    Places: 5  
    Descapotable: No  
    Propietari: Clara
```



□ **CC BY 4.0**

Contingut elaborat per Carles Bonet Papell i Cristina Reina León.

<https://creativecommons.org/licenses/by/4.0/>